LA FRUCTOSA · DOCUMENTACIÓN TÉCNICA

Master Prompt: Documentación Técnica y Arquitectura de “La Fructosa”

Instrucciones para la IA (Claude / GPT-4 / Cursor): Utiliza este documento como la especificación técnica completa y la fuente de verdad (Single Source of Truth) del proyecto **La Fructosa**. Este documento define la arquitectura de software, el modelo de datos, la seguridad Row Level Security (RLS), la lógica de conversión multimoneda y la especificación del frontend de la Progressive Web App (PWA).

1. Resumen Ejecutivo del Proyecto

- **Nombre de la PWA:** La Fructosa
- **Propósito:** Gestión financiera privada, segura, multitenant y multimoneda para uso familiar en distintas regiones (Venezuela, España, EE. UU.).
- **URL de Producción:** `https://lfzv98.github.io/la-fructosa/`
- **Infraestructura Frontend:** PWA ligera (HTML5, Tailwind CSS / Custom CSS, Vanilla JS) alojada en GitHub Pages.
- **Infraestructura Backend:** Supabase (PostgreSQL, GoTrue Auth, Row Level Security).
- **Estética Visual:** Paleta de colores pastel (Azul Pastel `#A0C4FF`, Amarillo Pastel `#FDFFB6`, Salmón Pastel `#FFADAD`).

2. Arquitectura de Seguridad (Row Level Security - RLS)

El principio fundamental del backend es el **aislamiento absoluto de datos por usuario**. Ningún usuario puede consultar, modificar o eliminar registros pertenecientes a otro usuario.

Principios RLS Aplicados:

1. **Autenticación:** Gestionada mediante `auth.users` de Supabase. Cada usuario posee un `UUID` único asignado por `auth.uid()`.
2. **Políticas SQL:** Todas las tablas de datos (`profiles`, `accounts`, `transactions`, `budgets`, `recurring_expenses`) obligan la regla:

POLICY: auth.uid()=user_id

3. **Privacidad Admin vs. User:** El cliente web solo consulta Supabase utilizando la `anon_key` pública, asegurando que las políticas RLS se ejecuten a nivel de motor PostgreSQL antes de entregar cualquier respuesta HTTP.

---

3. Modelo de Datos Relacional (PostgreSQL / Supabase)

Script SQL Completo de Inicialización (`schema.sql`):

```
-- 1. TABLA DE PERFILES DE USUARIO
CREATE TABLE IF NOT EXISTS public.profiles (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    email TEXT NOT NULL,
    full_name TEXT,
    base_currency TEXT DEFAULT 'USD' CHECK (base_currency IN ('USD', 'VES', 'EUR')),
    region TEXT DEFAULT 'VE' CHECK (region IN ('VE', 'US', 'ES')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view own profile"
    ON public.profiles FOR SELECT USING (auth.uid() = id);
CREATE POLICY "Users can update own profile"
    ON public.profiles FOR UPDATE USING (auth.uid() = id);
CREATE POLICY "Users can insert own profile"
    ON public.profiles FOR INSERT WITH CHECK (auth.uid() = id);

-- 2. TABLA DE CUENTAS / BILLETERAS
CREATE TABLE IF NOT EXISTS public.accounts (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
    name TEXT NOT NULL,
    account_type TEXT NOT NULL, -- Zelle, Efectivo, Pago Móvil, Wire, USDT, etc.
    currency TEXT NOT NULL CHECK (currency IN ('USD', 'VES', 'EUR', 'USDT')),
    initial_balance NUMERIC(15, 2) DEFAULT 0.00 NOT NULL,
    current_balance NUMERIC(15, 2) DEFAULT 0.00 NOT NULL,
    color TEXT DEFAULT '#A0C4FF',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.accounts ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can manage own accounts"
    ON public.accounts ALL USING (auth.uid() = user_id);

-- 3. TABLA DE TRANSACCIONES / MOVIMIENTOS
CREATE TABLE IF NOT EXISTS public.transactions (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
    account_id UUID REFERENCES public.accounts(id) ON DELETE CASCADE NOT NULL,
    destination_account_id UUID REFERENCES public.accounts(id) ON DELETE SET NULL, -- Para
transferencias
    type TEXT NOT NULL CHECK (type IN ('income', 'expense', 'transfer')),
    category TEXT NOT NULL,
    amount NUMERIC(15, 2) NOT NULL,
    currency TEXT NOT NULL CHECK (currency IN ('USD', 'VES', 'EUR', 'USDT')),
    exchange_rate NUMERIC(15, 4) DEFAULT 1.0000,
```

```
    description TEXT,
    transaction_date TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.transactions ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can manage own transactions"
    ON public.transactions ALL USING (auth.uid() = user_id);

-- 4. TABLA DE PRESUPUESTOS MENSUALES
CREATE TABLE IF NOT EXISTS public.budgets (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
    category TEXT NOT NULL,
    monthly_limit NUMERIC(15, 2) NOT NULL,
    currency TEXT DEFAULT 'USD' CHECK (currency IN ('USD', 'VES', 'EUR')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.budgets ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can manage own budgets"
    ON public.budgets ALL USING (auth.uid() = user_id);

-- 5. TABLA DE GASTOS RECURRENTES / SUSCRIPCIONES
CREATE TABLE IF NOT EXISTS public.recurring_expenses (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
    account_id UUID REFERENCES public.accounts(id) ON DELETE CASCADE NOT NULL,
    name TEXT NOT NULL,
    amount NUMERIC(15, 2) NOT NULL,
    currency TEXT NOT NULL CHECK (currency IN ('USD', 'VES', 'EUR', 'USDT')),
    category TEXT NOT NULL,
    due_day INTEGER CHECK (due_day BETWEEN 1 AND 31),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.recurring_expenses ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can manage own recurring expenses"
    ON public.recurring_expenses ALL USING (auth.uid() = user_id);
```

---

4. Adaptación Regional y Sincronización Financiera

Permutaciones por Región de Usuario:

1. **Región 🇪🇸 España (`ES`):**
    - **Moneda Base:** EUR (€).
    - **Comportamiento Visual:** Oculta indicadores de brecha cambiaria y tasas BCV/USDT.
    - **Tipos de Cuentas:** Banco Español, Efectivo EUR, Bizum, Tarjetas.
2. **Región 🇺🇸 Estados Unidos (`US`):**
    - **Moneda Base:** USD ($).

- **Comportamiento Visual:** Enfocado exclusivamente en Dólares. Oculta indicadores de tasas paralelas.
- **Tipos de Cuentas:** Bank of America, Zelle, Wire Transfer, Cash, Credit Card.

3. **Región 🇻🇪 Venezuela (`VE`):**
   - **Moneda Base:** Multimoneda (VES / USD / USDT / EUR).
   - **Comportamiento Visual:** Despliega barra superior con indicadores en tiempo real:
     - **Tasa BCV Oficial:** Obtenida vía API pública (`pydolarve`).
     - **Tasa Euro BCV:** Tasa oficial en EUR.
     - **Tasa USDT / Binance P2P:** Tasa paralela/mercado digital.
   - **Consolidación:** Convierte automáticamente los saldos de Pago Móvil y cuentas en VES a USD o moneda base elegida.

---

5. Estructura de Archivos del Frontend

```
la-fructosa/
├── index.html                  # Contenedor PWA, maquetación HTML5 y layouts modales
├── manifest.webmanifest        # Configuración de instalación PWA
├── favicon.png                 # Ícono web con fondo pastel
├── sw.js                       # Service Worker para almacenamiento en caché local
├── css/
│   └── styles.css              # Variables CSS con paleta pastel, componentes y utilidades
└── js/
    ├── config.js               # Parámetros de inicialización de Supabase
    ├── supabase.js             # Métodos Wrapper para Supabase Client JS (CRUD & Auth)
    ├── money.js                # Motor de conversión, cálculo de balances y Fetch de APIs de tasas
    ├── api.js                  # Manejo de peticiones de tasas (BCV, Euro, USDT)
    ├── export.js               # Generador de reportes CSV / Excel
    └── app.js                  # Lógica de navegación SPA, eventos DOM y renderizado
```

---

6. Paleta de Colores y Guía de Estilos UI/UX

```css
:root {
  /* Paleta Pastel Principal */
  --color-blue-pastel: #A0C4FF;     /* Estructura, navegación, fondos de tarjetas */
  --color-yellow-pastel: #FDFFB6;   /* Badges de moneda, alertas de presupuesto, tasas */
  --color-salmon-pastel: #FFADAD;   /* Botones primarios, registros de gastos, acciones principales
*/

  /* Variantes de Apoyo */
  --color-bg-dark: #121824;         /* Fondo en modo oscuro */
  --color-card-dark: #1E2640;       /* Superficie de tarjetas en modo oscuro */
  --color-text-light: #F8FAFC;      /* Texto sobre superficies oscuras */
  --color-text-dark: #1E293B;       /* Texto sobre elementos pastel */
}
```

---

7. Instrucciones para la IA (Mantenimiento y Evolución)

1. **Preservar el aislamiento de datos:** Nunca modificar las consultas a Supabase omitiendo el filtro o política `auth.uid() = user_id`.
2. **Transferencias sin doble contabilización:** Toda transferencia entre dos cuentas del mismo usuario debe descontar del origen y sumar al destino mediante una única transacción lógica con categoría `Transferencia Interna`.
3. **Single-Page Application (SPA):** Toda la interacción debe realizarse dinámicamente sin recargar la página (`preventDefault()` en formularios).